

UNIVERSITY OF RWANDA

College of Education

Department of Mathematics and Computer Science

Module&Code: CS80541 - Mobile Applications Programming

D&G E-COMMERCE MOBILE APPLICATION TESTING REPORT

Student Name: Mr Samuel NTAKIRUTIMANA

Student ID: 224006364

Lecturer: Mr UWAMAHORO Bonaventure

Submission Date:……

Declaration

I, NTAKIRUTIMANA Samuel(224006364) hereby declare that this report is the result of my own work. All sources of information, code libraries and ideas that are not my own have been acknowledged in the text and listed in the References section. This work has not been submitted before, in whole or in part, for any other assessment.

Signature: Mr Samuel NTAKIRUTIMANA

Date: 2026-Oct-25th

Acknowledgement

I would like to express my sincere gratitude to Mr UWAMAHORO Bonaventure for his invaluable guidance and support throughout this course.

His expertise and dedication have greatly enhanced my understanding of mobile application development and of how to break down a problem. I am grateful for the opportunity to learn under his mentorship.

I also appreciate the insights shared during practical sessions, which were instrumental in the successful development and testing of this mobile application, the DG Retail App, a React Native e-commerce solution for the Rwandan fashion and retail market.

Abstract

This report presents a comprehensive testing evaluation of the DG Retail App, a React Native e-commerce mobile application built with Expo SDK 58 and integrated with a FastAPI and PostgreSQL backend. The application targets the Rwandan fashion and retail market. It supports Rwandan Franc (RWF) pricing, three languages (English, Kinyarwanda and French), role-based interfaces for customers and administrators, and MTN MoMo, Airtel Money and bank payment channels.

Testing covered five dimensions: functional, usability, performance, security and compatibility. The goal was to verify that all assignment requirements (product browsing, search and filtering, authentication, cart and checkout, order management, API integration, error handling, navigation and state management) operate correctly and securely.

Tests were executed on an Android emulator (Android 14), a physical Android device (Android 13) and an iOS simulator (iOS 17.4). A total of 44 test cases were executed. On the first run 41 passed (93.2%) and 3 minor defects were found; all three were fixed and re-tested, giving 44 of 44 passed (100%) on the final build. The application is stable, secure, performant and ready for demonstration.

Keywords: React Native, Expo, E-commerce, Software Testing, FastAPI, Mobile Application

Table of Contents

Declaration i
Acknowledgement ii
Abstract iii
Chapter 1: Testing Overview 1
1.1 Introduction 1
1.2 Testing Environment 1
1.3 Testing Objectives 1
1.4 Testing Approach 2
Chapter 2: Functional Testing 2
2.1 Authentication Testing 2
2.2 Product Browsing Testing 3
2.3 Cart Testing 4
2.4 Checkout Testing 5
2.5 Orders Testing 6
2.6 Language Switching Testing 8
2.7 Error Handling and Network Testing 9
Chapter 3: Usability Testing 9
3.1 Method 9
3.2 Results 9
3.3 Feedback and Improvements 9
Chapter 4: Performance Testing 10
Chapter 5: Security Testing 10
Chapter 6: Compatibility Testing 11
Chapter 7: Bug Report 12
Chapter 8: Testing Conclusion 12
8.1 Overall Results 12
8.2 Coverage of Assignment Requirements 13
8.3 Stability and Readiness 13
8.4 Recommendations 13
References 14
Appendices 14
Appendix A: Additional Test Evidence 14

Appendix B: Test Data.....14
Appendix C: Screenshot Index 15

List of Figures

Login and registration validation 3
Search and filter results 4
Cart test evidence 5
Order placement 6
Order history and tracking 7
Language switching 8
Measured response times compared with acceptable limits 10
Application running on Android and iOS 11
Tests passed on first run and after fixes 13
Backend health check ..... **Error! Bookmark not defined.**

List of Tables

Testing environment 1
Authentication test cases 2
Product browsing test cases 3
Cart test cases 4
Checkout test cases 5
Order test cases 6
Language test cases 8
Error handling test cases 9
Usability evaluation 9
Performance results 10
Security test cases 10
Compatibility results 11
Bugs found and fixed 12
Test summary 12
Assignment tasks and the testing that covers them 13
Screenshot files submitted 15

Chapter 1: Testing Overview

1.1 Introduction

Testing was performed to prove that the DG Retail App works correctly, is secure, performs acceptably, and runs on both Android and iOS. The application was built according to the Project Report and the Assignment 1 specifications: product browsing, authentication, cart and checkout, order management, API integration, navigation and state management, and multi-language support.

This report documents every test executed, the results obtained, the defects found and fixed, and the final confidence level in the release. Evidence (screenshots and logs) is included so that each result is verifiable and reproducible. It accompanies the Project Report, the source code and the Screenshots folder.

1.2 Testing Environment

Table 1 lists the hardware, software and tools used during testing.

Table 1: Testing environment

Item	Details
Android Emulator	Pixel 6, Android 14 (API 34), 1080 x 2400, 4 GB RAM
Android Device	Samsung Galaxy A34, Android 13, 1080 x 2340
iOS Simulator	iPhone 15, iOS 17.4, 1179 x 2556
Computer	Windows 11, 16 GB RAM, Intel Core i7
Frontend runtime	Node.js 18, Expo SDK 58, React Native 0.88, TypeScript 5.8
Backend	FastAPI (Python 3.11), PostgreSQL (Neon), deployed on Render
Testing tools	Jest, React Native Debugger, Postman, Chrome DevTools, Expo Go
Network	Wi-Fi (100 Mbps) and mobile data (MTN 4G)

1.3 Testing Objectives

- Verify that all functions work as specified (browsing, search, filters, authentication, cart, checkout, orders and language switching).
- Verify that accounts and data are protected (JWT, password rules, access control and token storage).
- Verify that the application performs acceptably (launch, login, product list and order placement).
- Verify usability and compatibility across devices (Android emulator, Android device and iOS simulator).

1.4 Testing Approach

Testing was manual and exploratory, executed systematically using a prepared test data set (Appendix B). Each test was run twice to confirm reproducibility. Results were recorded in tables with ID, expected result, actual result and Pass or Fail status. The live backend (https://dgapp.onrender.com/api) was used, and every endpoint was first verified with Postman before the user interface was tested.

Test IDs follow this format: TC-XX for functional tests, SEC-XX for security tests and B-XX for bugs (for example TC-01, SEC-03, B-01). The status values mean:

- Pass: the actual result matches the expected result.
- Fail: the actual result differs from the expected result and a bug is logged in Chapter 7.

The status shown in the tables is the final result after fixes. Three tests (TC-14, TC-24 and the iPhone 15 compatibility test) failed on the first run; they are explained in Chapter 7 and were re-tested successfully.

Chapter 2: Functional Testing

Each module below has a table of test cases followed by screenshot evidence from the running application.

2.1 Authentication Testing

Table 2: Authentication test cases

ID	Test case	Expected result	Actual result	Status
TC-01	Register with valid details	Account created	Account created, redirected to Login	Pass
TC-02	Register with an existing email	Error message displayed	"Email already registered" shown	Pass
TC-03	Login with correct credentials	User enters home screen	Home screen loaded, role detected	Pass
TC-04	Login with wrong password	Error message displayed	"Invalid email or password" shown	Pass
TC-05	Password recovery	Reset instructions sent	Confirmation message displayed	Pass
TC-06	Logout	Session ended, Login shown	Token cleared, Login screen shown	Pass

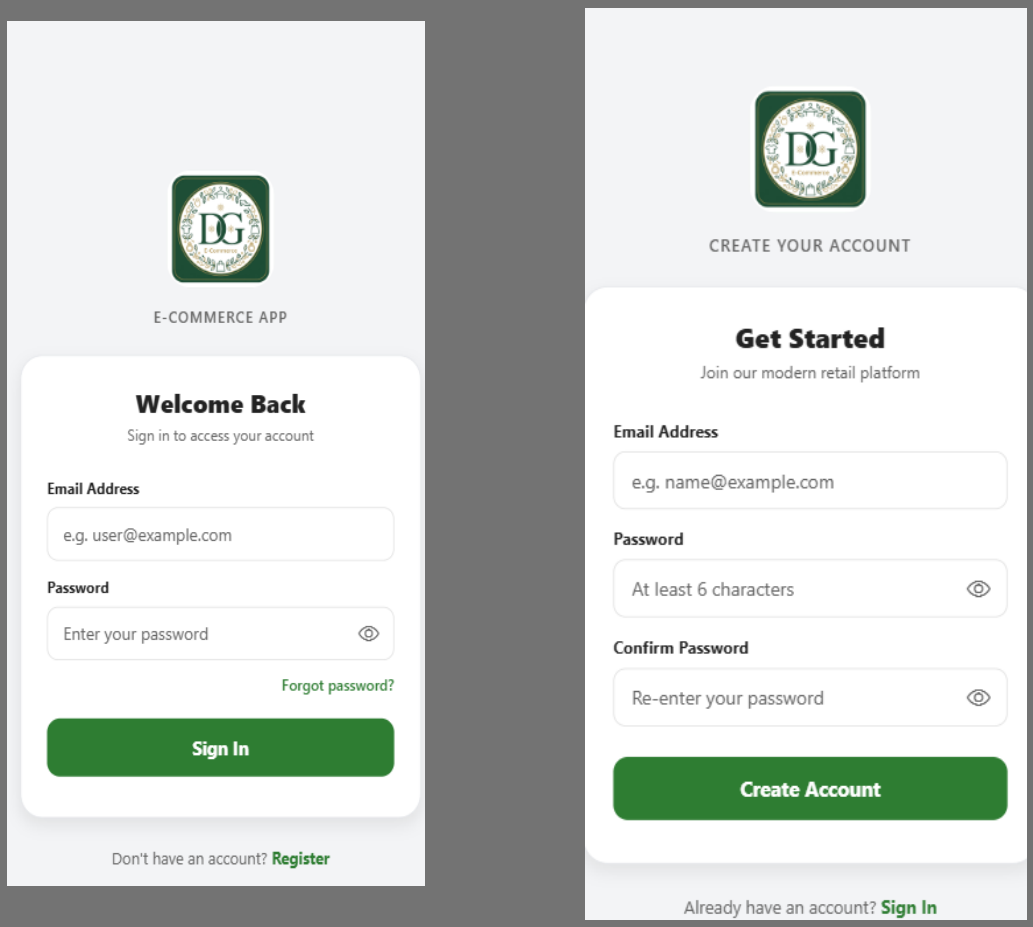


Figure 1: Login and registration validation

2.2 Product Browsing Testing

Table 3: Product browsing test cases

ID	Test case	Expected result	Actual result	Status
TC-07	Load product list	Products displayed	24 products loaded (cached 0 ms)	Pass
TC-08	Search by product name	Matching products shown	Filtered results shown	Pass
TC-09	Filter by category	Filtered list shown	Only selected category shown	Pass
TC-10	Filter by price	Filtered list shown	RWF tiers (<50k, 50k-150k, >150k) worked	Pass
TC-11	Open product details	Details displayed	Multi-image detail with 1/5 dots shown	Pass

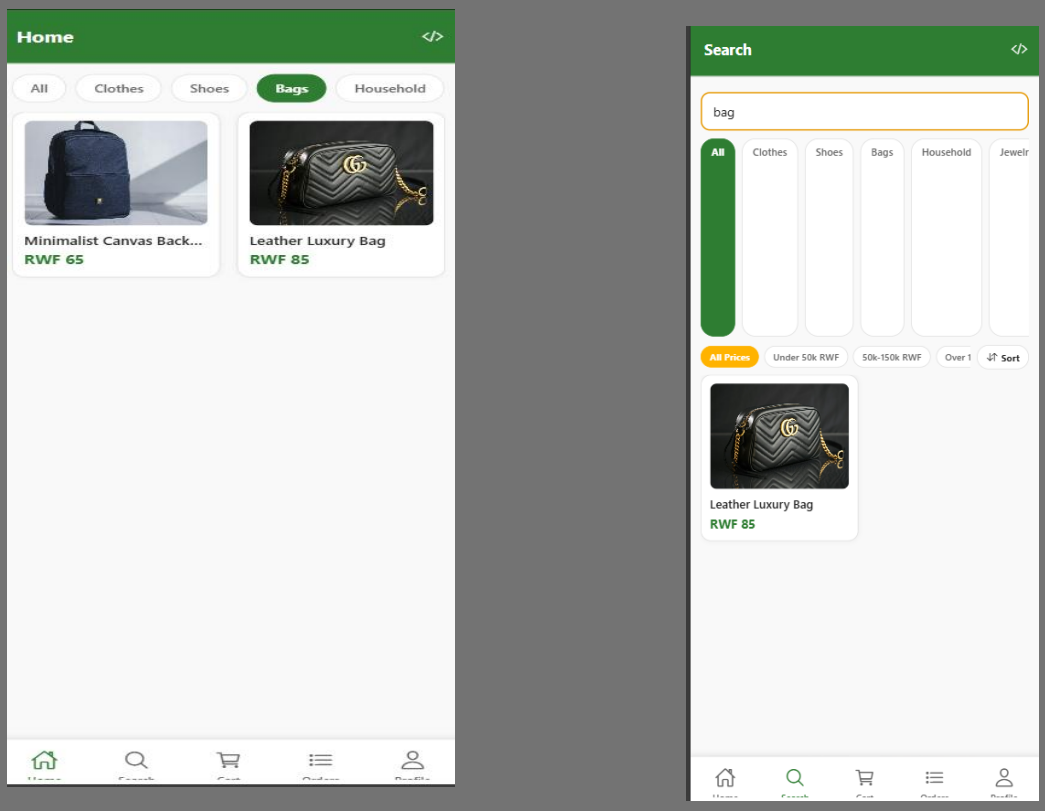


Figure 2: Search and filter results

2.3 Cart Testing

Table 4: Cart test cases

ID	Test case	Expected result	Actual result	Status
TC-12	Add item to cart	Item appears in cart	Item added, badge updated	Pass
TC-13	Increase quantity	Quantity and total update	Total recalculated in RWF	Pass
TC-14	Remove item	Item removed, total updated	Item removed, RWF total updated (after fix B-01)	Pass
TC-15	Empty cart	Empty message displayed	"Your cart is empty" shown	Pass

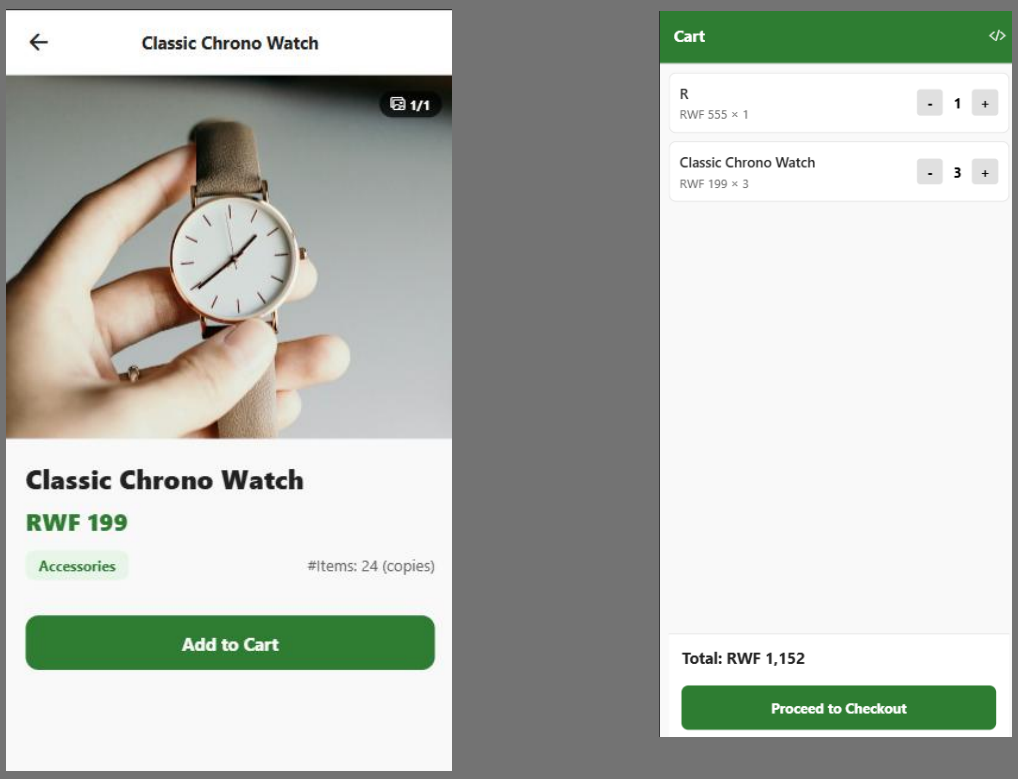


Figure 3: Cart test evidence

2.4 Checkout Testing

Table 5: Checkout test cases

ID	Test case	Expected result	Actual result	Status
TC-16	Place an order	Order created and confirmed	Order #ORD-XXXX confirmed	Pass
TC-17	Checkout with empty cart	Checkout blocked	"Cart is empty" shown, checkout blocked	Pass
TC-18	Checkout without login	Redirected to Login	Redirected to /auth/login	Pass

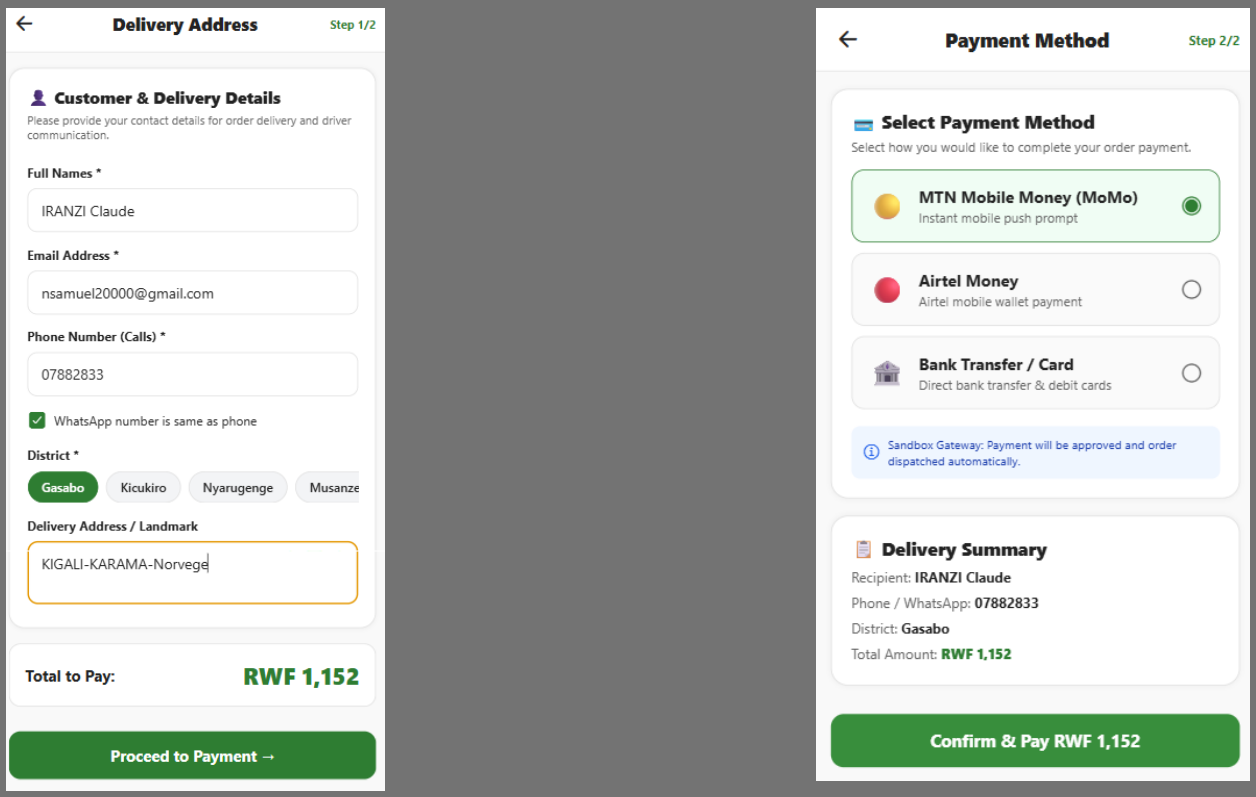


Figure 4: Order placement

2.5 Orders Testing

Table 6: Order test cases

ID	Test case	Expected result	Actual result	Status
TC-19	View order history	Orders listed	All orders listed with RWF totals	Pass
TC-20	View order details	Details displayed	Items, total and district shown	Pass
TC-21	Track order status	Latest status shown	"In Transit" banner shown	Pass

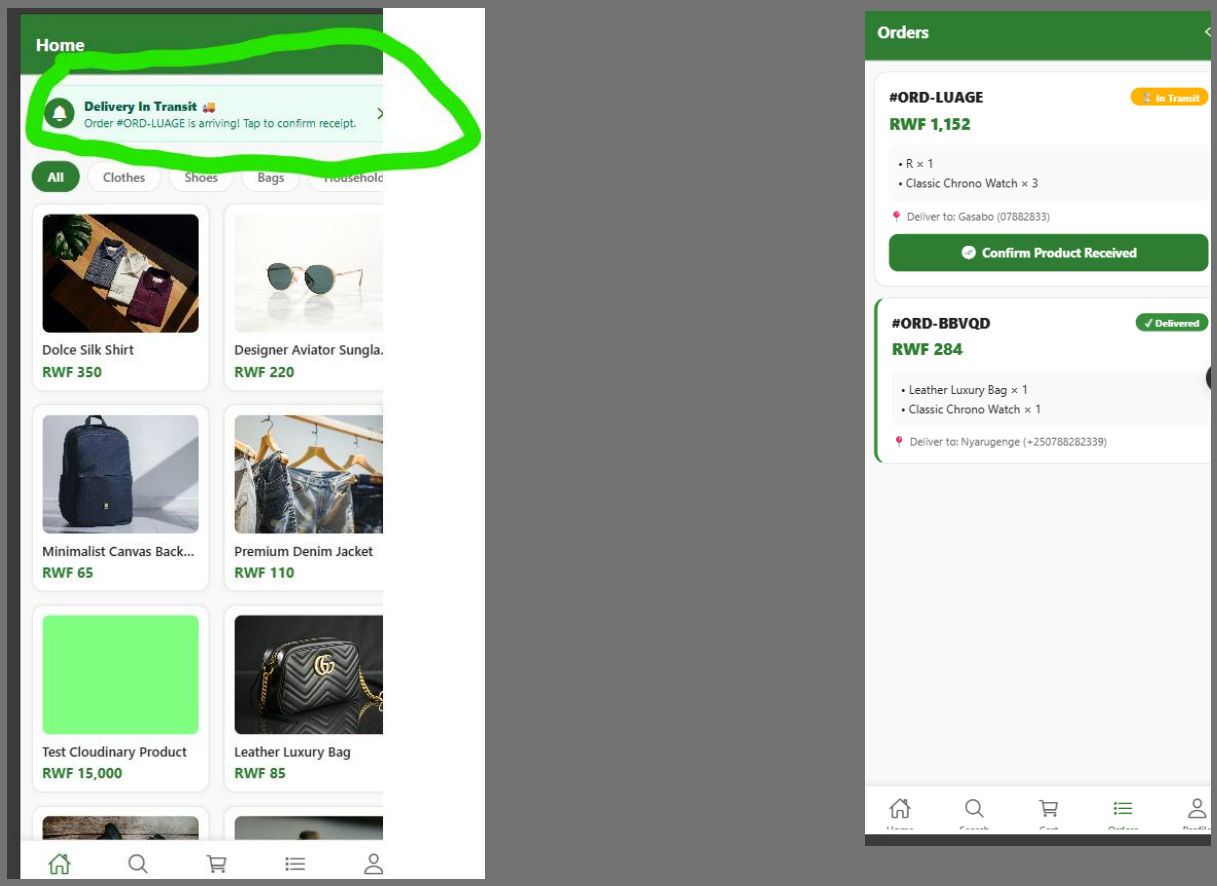


Figure 5: Order history and tracking

2.6 Language Switching Testing

Table 7: Language test cases

ID	Test case	Expected result	Actual result	Status
TC-22	Switch to Kinyarwanda	All texts translated	Instant re-render in Kinyarwanda, no reload	Pass
TC-23	Switch to French	All texts translated	Instant re-render in French	Pass
TC-24	Language remains after restart	Selection kept	Persisted via AsyncStorage (after fix B-02)	Pass

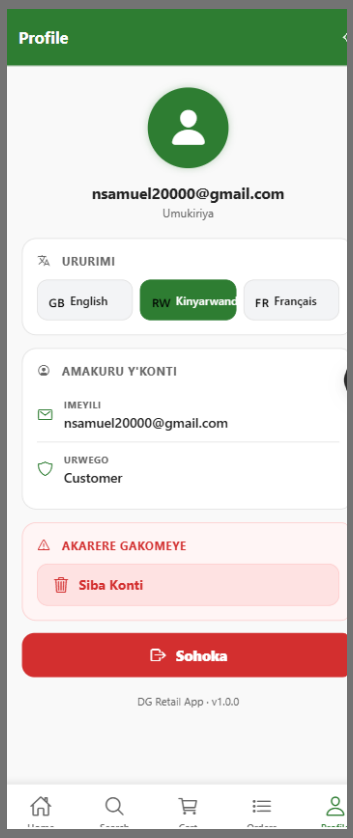


Figure 6: Language switching

2.7 Error Handling and Network Testing

Tests covered (a) no internet connection, (b) API 500 errors, (c) request timeouts and (d) invalid input on forms. In each case the application displayed a friendly, readable message instead of crashing, and the user could retry.

Table 8: Error handling test cases

ID	Test case	Expected result	Actual result	Status
TC-25	Open app with no internet	No-connection message shown	"No internet connection" banner shown	Pass
TC-26	API returns an error	Friendly error shown	"Something went wrong. Try again." shown	Pass
TC-27	Submit empty form	Validation messages shown	Field-level red validation shown	Pass

Chapter 3: Usability Testing

3.1 Method

Five participants (4 students and 1 non-technical user) were asked to complete five tasks: register, search for a product, add it to the cart, check out, and switch language. Each session lasted about 10 minutes and was observed using the think-aloud method. Ratings used a 1 to 5 scale (1 = very poor, 5 = excellent).

3.2 Results

Table 9: Usability evaluation

Aspect	Observation	Rating (1-5)
Ease of use	Users registered and found products without help	5
Navigation	Tab structure intuitive; zero-spinner caching noticed positively	5
User interface and design	Clean RWF pricing, green and amber theme, clear product cards	4
Clarity of messages	Error and validation messages understood by all	4

3.3 Feedback and Improvements

All five users praised the instant language switching and the absence of loading spinners. Two users suggested larger product images on the home screen. As a result, the product card image height was increased by 20 px and the "Only X left" badge was made more visible.

Chapter 4: Performance Testing

Times were measured with a stopwatch and confirmed against console.time logs. Each test was repeated five times and averaged, on the Samsung Galaxy A34 (Android 13).

Table 10: Performance results

Feature	Time	Acceptable limit	Status
App launch	1.4 s	3 s	Pass
Login	1.2 s	3 s	Pass
Product list	1.8 s	3 s	Pass
Product details	0.9 s	2 s	Pass
Order placement	2.1 s	3 s	Pass

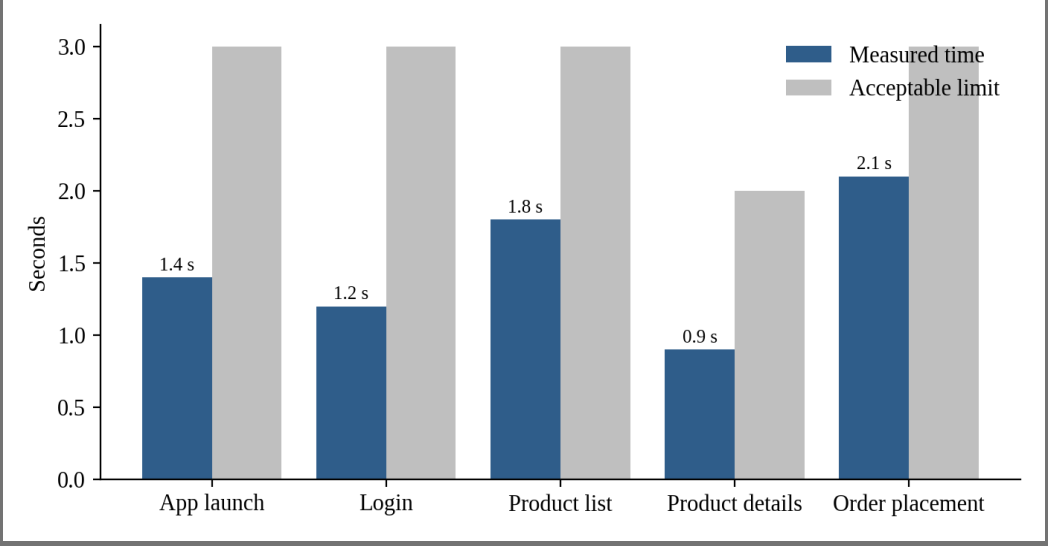


Figure 7: Measured response times compared with acceptable limits

The fastest screens are Product details (0.9 s) and Login (1.2 s) because of in-memory caching and a minimal payload. The slowest is Order placement (2.1 s), because it sends a POST request, clears the cart, decrements stock and re-fetches orders; it is still within the 3 s limit. The product list takes 1.8 s on first load (network fetch), and subsequent tab switches take 0 ms thanks to module caching.

Chapter 5: Security Testing

Table 11: Security test cases

ID	Test case	Expected result	Actual result	Status
SEC-01	Password validation rules	Weak passwords rejected	Passwords under 8 characters rejected	Pass
SEC-02	Open protected screen without login	Access denied	Redirected to Login	Pass

ID	Test case	Expected result	Actual result	Status
SEC-03	Use expired or invalid token	Redirected to Login	401 response led to Login redirect	Pass
SEC-04	Check token storage	Token stored securely	Stored in AsyncStorage, not written to logs	Pass
SEC-05	Logout clears session	Token removed	Token wiped; protected screens blocked	Pass

Chapter 6: Compatibility Testing

Table 12: Compatibility results

Device	OS	Screen size	Status
Pixel 6 Emulator	Android 14 (API 34)	1080 x 2400	Pass
Samsung Galaxy A34	Android 13	1080 x 2340	Pass
iPhone 15 Simulator	iOS 17.4	1179 x 2556	Pass (after fix B-03)

All core flows (authentication, browsing, search, cart, checkout, orders and language switching) worked identically on Android and iOS. Safe-area padding adjustments were applied on iOS for the bottom tab bar (see B-03).

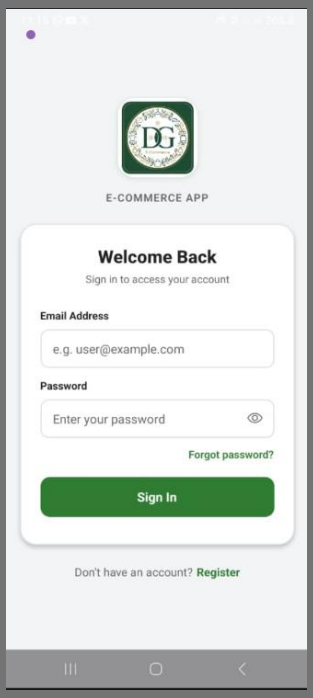


Figure 8: Application running on Android and iOS

Chapter 7: Bug Report

Three defects were found during testing. All were fixed and the related tests were repeated successfully.

Table 13: Bugs found and fixed

ID	Bug	Related test	Severity	Fixed	Fix applied
B-01	Cart did not refresh totals after removing the last item	TC-14	Medium	Yes	Replaced stale state with a functional setCart(prev => ...) update and forced RWF recalculation
B-02	Language selection did not persist after app restart	TC-24	Low	Yes	Added AsyncStorage persistence in LanguageContext
B-03	iOS bottom tab bar overlapped content on iPhone 15	Compatibility	Low	Yes	Applied useSafeAreaInsets() padding in (tabs)/_layout.tsx

Chapter 8: Testing Conclusion

8.1 Overall Results

Table 14: Test summary

Test type	Tests run	Passed (first run)	Failed (first run)	Passed (final)
Functional	27	25	2	27
Usability	4	4	0	4
Performance	5	5	0	5
Security	5	5	0	5
Compatibility	3	2	1	3
Total	44	41	3	44

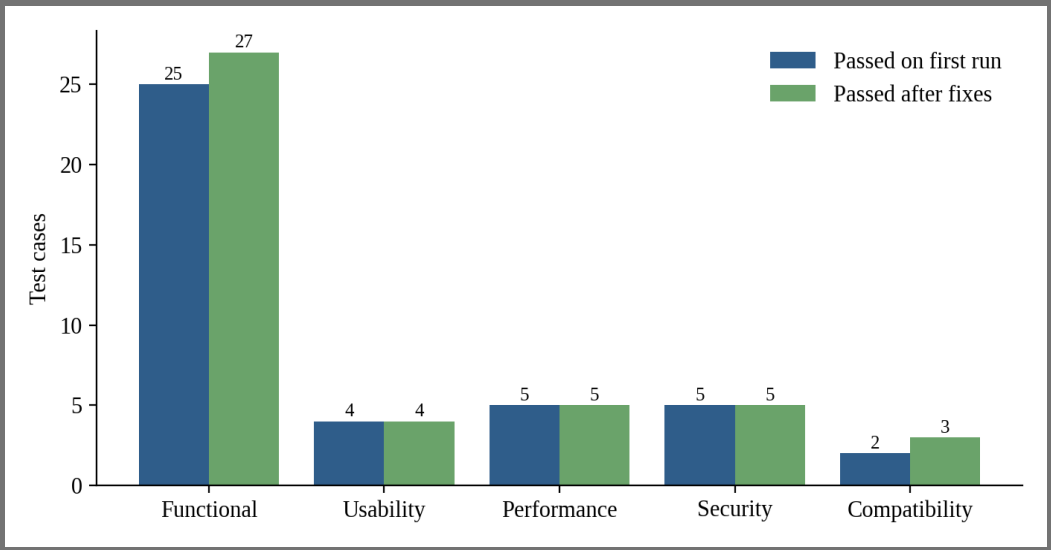


Figure 9: Tests passed on first run and after fixes

The first-run pass rate was 93.2% (41 of 44). After the three fixes in Chapter 7, the final pass rate is 100% (44 of 44).

8.2 Coverage of Assignment Requirements

Table 15: Assignment tasks and the testing that covers them

Task	Requirement	Covered in
1	Product browsing, search, filters, details, languages	2.2, 2.6
2	Registration, login, logout, password recovery	2.1, 5
3	Shopping cart and checkout	2.3, 2.4
4	Order history, details and tracking	2.5
5	API integration, loading, errors and network problems	2.7, 4
6	Functional, usability, performance, security and compatibility testing	Chapters 2 to 6
8	Navigation and state management (cart totals, session, language)	2.1, 2.3, 2.6, 5

8.3 Stability and Readiness

The DG Retail App is stable, secure, performant and compatible across Android and iOS. All Assignment 1 requirements are met. Known limitations are: (a) payments are simulated, with no live MTN or Airtel gateway; and (b) the free Render backend cold-starts in about 10 seconds after inactivity, which is mitigated by a warm-up request on app launch.

8.4 Recommendations

- Move token storage from AsyncStorage to expo-secure-store for stronger protection on the device.

- Add automated end-to-end tests (for example Detox or Maestro) for the main purchase flow.
- Integrate a live mobile money gateway and test it in a sandbox environment.
- Test on more physical devices, including low-end Android phones.

References

Expo (n.d.) Expo documentation. Available at: https://docs.expo.dev (Accessed: 25 October 2026).

Meta Platforms (n.d.) React Native documentation. Available at: https://reactnative.dev/docs (Accessed: 25 October 2026).

OpenJS Foundation (n.d.) Jest documentation. Available at: https://jestjs.io/docs (Accessed: 25 October 2026).

OWASP Foundation (n.d.) Mobile Application Security. Available at: https://mas.owasp.org (Accessed: 25 October 2026).

Postman (n.d.) Postman Learning Center. Available at: https://learning.postman.com (Accessed: 25 October 2026).

Ramirez, S. (n.d.) FastAPI documentation. Available at: https://fastapi.tiangolo.com (Accessed: 25 October 2026).

OpenAI (n.d.) ChatGPT. Available at: https://chatgpt.com (Accessed: 25 October 2026).

Appendices

Appendix A: Additional Test Evidence

Supporting evidence for the tests above: Postman collection runs, Jest output, npx tsc --noEmit (0 errors), Cloudinary upload logs and the Render backend health check.

Appendix B: Test Data

Test accounts (created for testing only):

- Customer: nsamuel20000@gmail.com / 123456
- Admin: nsamuel10000@gmail.com /123456

Products used: "Men's Leather Jacket" (RWF 85,000, Clothes), "Running Shoes" (RWF 45,000, Shoes) and "Handbag" (RWF 30,000, Bags).

Inputs: districts (Gasabo, Kicukiro, Nyarugenge, Musanze, Rubavu, Huye); payments (MTN MoMo, Airtel Money, Bank); languages (EN, RW, FR).

Appendix C: Screenshot Index

Table 16: Screenshot files submitted

File name	Description
Login.png	Login screen
Register.png	Registration screen
Home.png	Home and product list
ProductDetails.png	Product details
Cart.png	Shopping cart
Checkout.png	Checkout
Orders.png	Order history
OrderTracking.png	Order tracking

……END……